Analysis and Design of Algorithms

CS3230

Greedy Algorithms Examples

- Activity selection
- Huffman code

A/P Lee Wee Sun

Activity Selection Problem

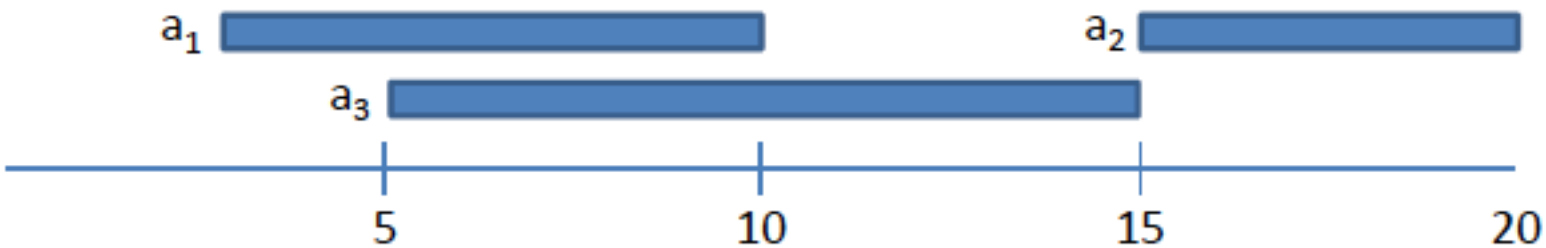
Given a set of activities $S = \{a_1, a_2, \ldots, a_n\}$

- Each activity a_i takes place during $[s_i, f_i)$
- Two activities $[s_i, f_i)$ and $[s_j, f_j)$ are compatible if they do not overlap, i.e., $s_i \geq f_j$ or $s_j \geq f_i$

Find a max-size subset of mutually **compatible** activities

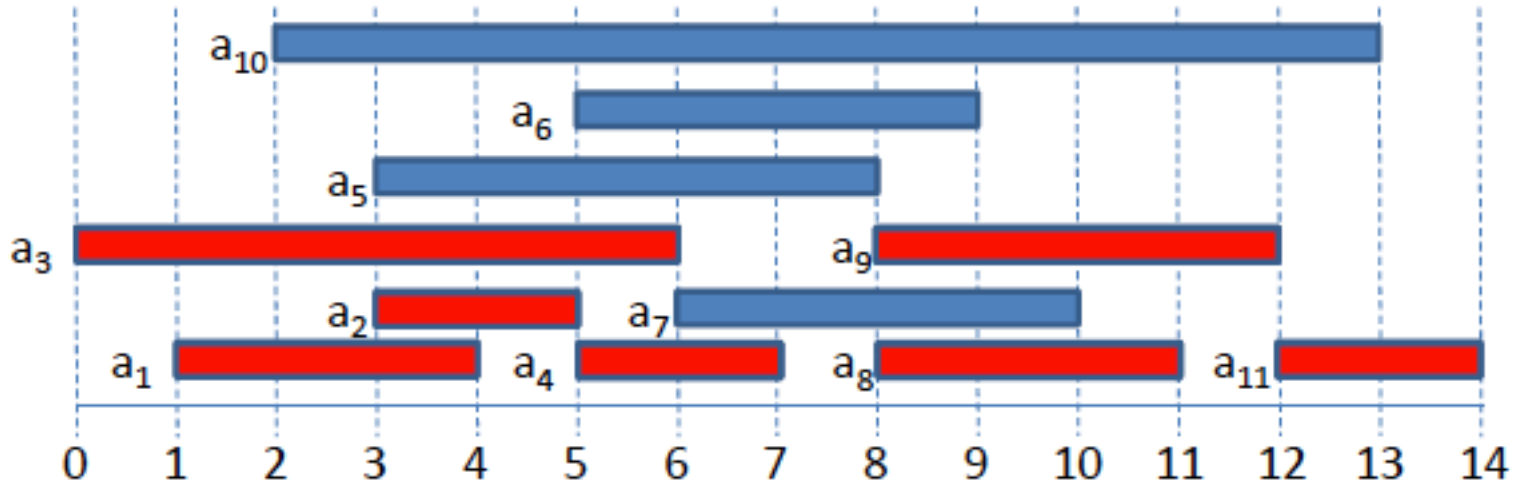
Example: a_1=[3, 10), a_2=[15, 20), a_3=[5, 15)

- $\{a_1$ and $a_2\}$ and $\{a_2$ and $a_3\}$ are compatible
- $\{a_1$ and $a_3\}$ are not compatible



Example: Find a max-size subset of mutually compatible activities

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14



$\{a_3, a_9, a_{11}\}$ are mutually compatible

$\{a_1, a_4, a_8, a_{11}\}$ is a largest set of mutually compatible activities

Another largest set is $\{a_2, a_4, a_9, a_{11}\}$

Optimal Substructure

Assume activities are sorted by the *finish time.*

Let S_{ij} be the subset of activities $a_k \in S$ that start after a_i finishes, and finish before a_j starts, i.e. $S_{ij} = \{a_k \in S : f_i \leq s_k < f_k \leq s_j\}$.

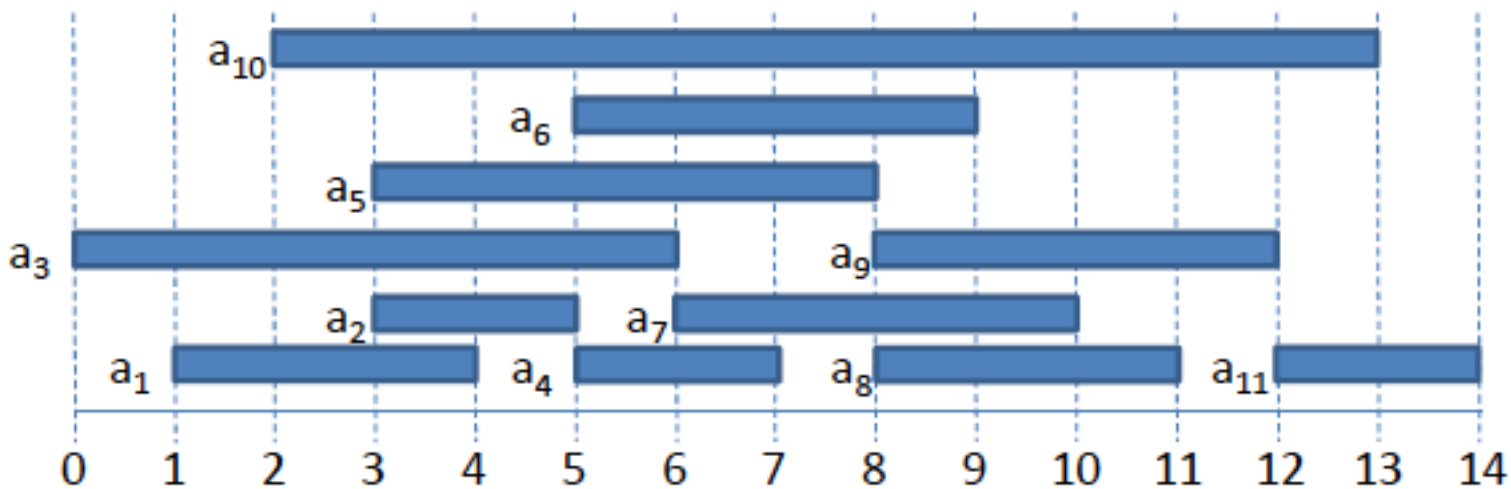
Add fictitious activities a_0 and a_{n+1} with $f_0 = 0$ and $s_{n+1} = \infty$ to simplify description.

Examples:

$S_{0,12} = \{a_1, a_2, \ldots, a_{11}\}$

$S_{1,11} = \{a_4, a_6, a_7, a_8, a_9\}$

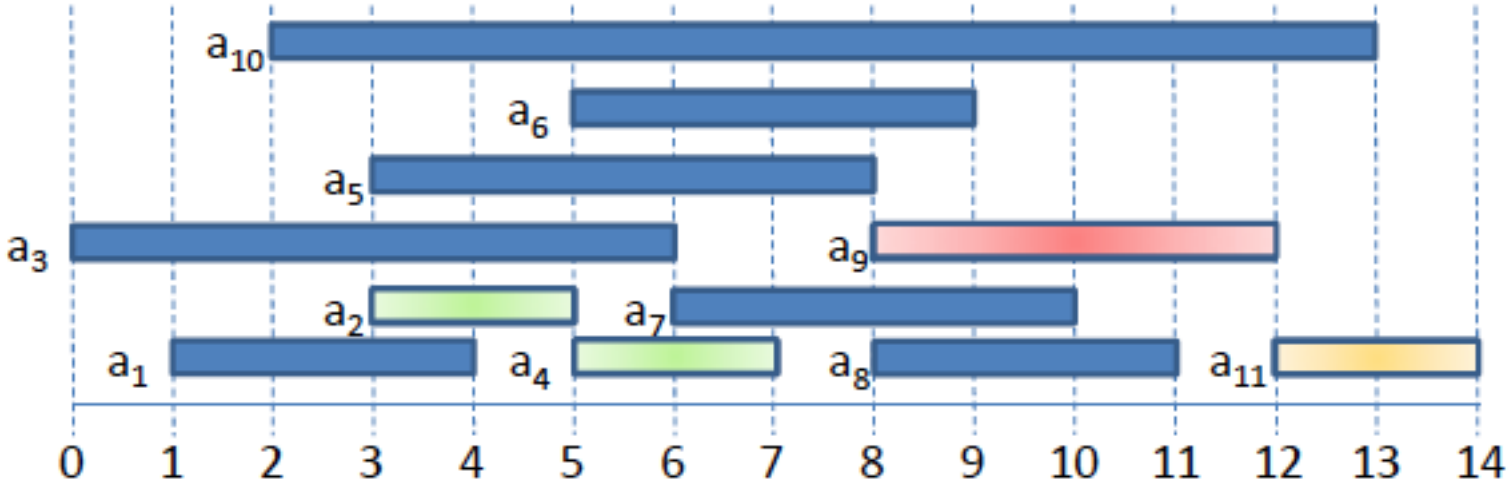
$S_{7,11} = \Phi$



Optimal substructure:

Let A_{ij} be an optimal solution for S_{ij}. Suppose $a_k \in A_{ij}$, then, $A_{ik} \cup \{a_k\} \cup A_{kj}$ is also an optimal solution for S_{ij}.

Example: An optimal solution for $S_{0,12}$ is $\{a_2, a_4, a_9, a_{11}\}$.
Consider $a_9 \in A_{0,12}$.
$A_{0,9}=\{a_2, a_4\}$ is an optimal solution for $S_{0,9}$ and $A_{9,12}=\{a_{11}\}$ is optimal solution for $S_{9,12}$. Then, $A_{0,9} \cup \{a_9\} \cup A_{9,12}$ is an optimal solution for $S_{0,12}$.



Recursive Formula

Let $c[i,j]$ be the maximum number of mutually compatible activities in S_{ij}.

Base case: $c[i,i] = 0$

$$c[i,j] = \max \begin{cases} 0 & \text{if } S_{ij} = \phi \\ \max_{i<k<j, a_k \in S_{ij}} \{c[i,k] + c[k,j] + 1\} & \text{if } S_{ij} \neq \phi \end{cases}$$

$O(n^3)$ dynamic programming

Greedy Choice

This problem satisfies an even more useful property:

Greedy-choice property:

A locally optimal choice is globally optimal

Intuition: The activity a_k in $S_{0,n+1}$ with the earliest finishing time is likely to be in the optimal solution.

In that case, A_{ik} in $A_{ik} \cup \{a_k\} \cup A_{kj}$ is empty, leaving one subproblem of finding $A_{kj.}$

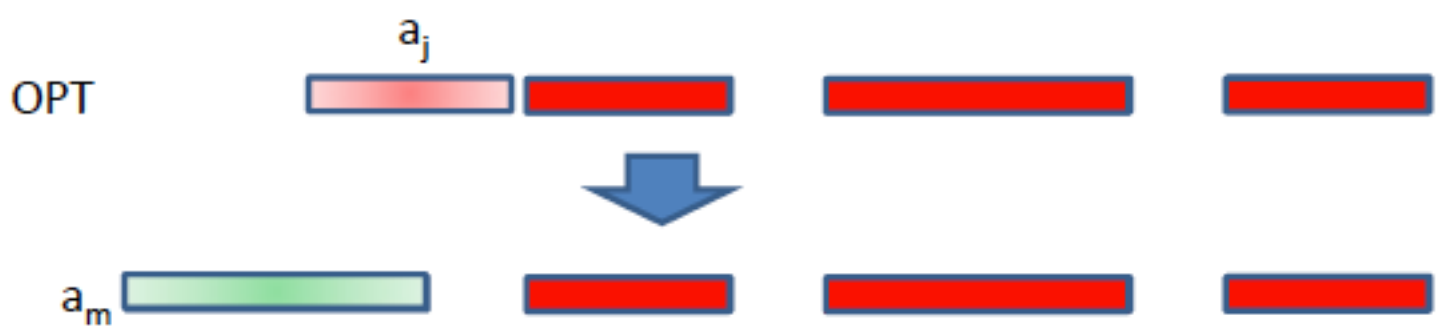
Consider a subproblem S_{ik} and let a_m be the activity with the *earliest finishing time.*

Lemma: There exists an optimal solution for S_{ik} that contains a_m.

Proof: (*Cut and Paste*)

Let OPT be an optimal solution for S_{ik}. Let a_j be the activity in OPT with the earliest finishing time.

- If $a_m = a_j$, then the lemma is clearly true
- Otherwise, $OPT - \{a_j\} \cup \{a_m\}$ is also an optimal solution for S_{ik} as a_m finished earlier than a_j



Standard technique for proving greedy choice:

Usually, we do the following (cut and paste):

1. Find the optimal choice
2. Replace the optimal choice by the greedy choice
3. Show that the greedy choice is as good as the optimal choice

GREEDY-ACTIVITY-SELECTOR(s,f)

$n \leftarrow \text{length}[s]$, $A \leftarrow \{a_1\}$, $i \leftarrow 1$

for $m \leftarrow 2$ **to** n

  do if $s_m \geq f_i$

    then $A \leftarrow A \cup \{a_m\}$

      $i \leftarrow m$

return A

Linear time, assuming activities already ordered in increasing finish time

GREEDY-ACTIVITY-SELECTOR(s,f)

$n \leftarrow$ length[s], $A \leftarrow \{a_1\}$, $i \leftarrow 1$

for $m \leftarrow 2$ **to** n

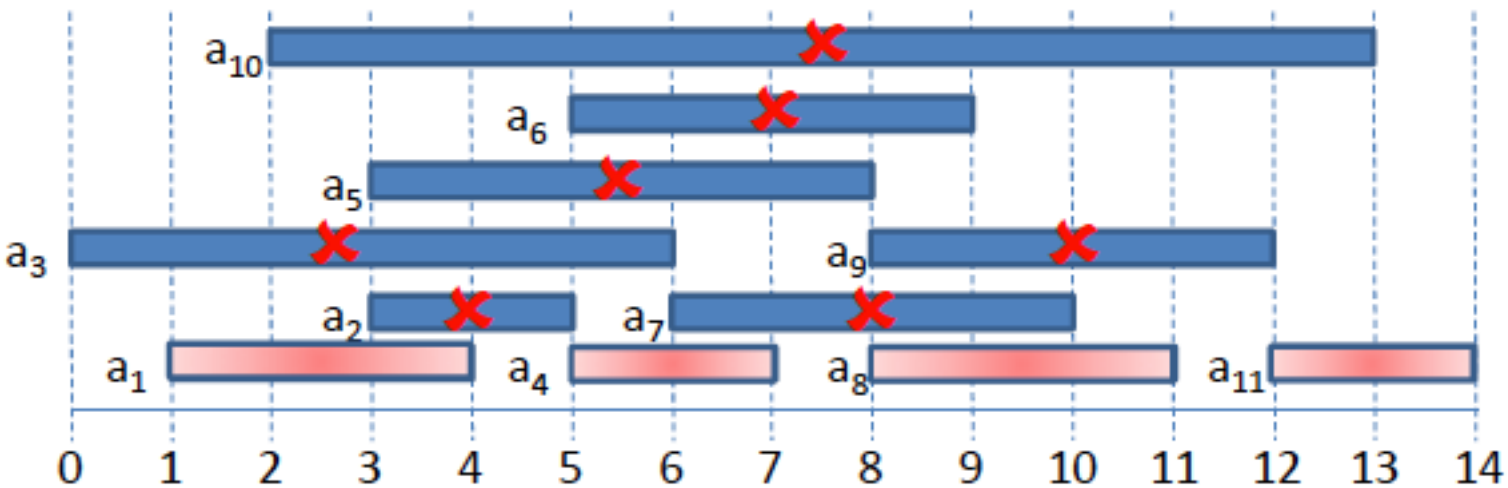
  do if $s_m \geq f_i$

    then $A \leftarrow A \cup \{a_m\}$

      $i \leftarrow m$

return A

Example: Greedy algorithm selects 4 activities: a_1, a_4, a_8, a_{11}



Greedy Algorithm Design Approach

1. Cast optimization problem as one where we make a choice and are left with one subproblem to solve.
2. Prove that there is an optimal solution that makes the greedy choice.
3. Show that if we combine the optimal solution to the subproblem to the greedy choice, we get the optimal solution (optimal substructure)

Analysis and Design of Algorithms

CS3230

Greedy Algorithms Examples

- Activity selection
- **Huffman code**

A/P Lee Wee Sun

Data Compression

Let S = "*ababcaad*", 8 characters

Fixed-length code to represent this sequence:

- The alphabet size is 4: $\{a, b, c, d\}$
- We can use 2 bits to represent each alphabet: $\{00, 01, 10, 11\}$

a	b	a	b	c	a	a	d
00	01	00	01	10	01	01	11

In total, it takes $8*2 = 16$ bits. Can we use less space?

Variable-length Code

Variable-length code may be shorter

We can use shorter code to represent more frequent alphabet

However, need to decode despite length of the different codeword being different…

Prefix code

Codes in which no codeword is the prefix of some other codewords

Example: $a = 10$, $p = 0$, $e = 1101$, $l = 111$, $i = 1100$

Decode "10001111101"

10-0-0-111-1101

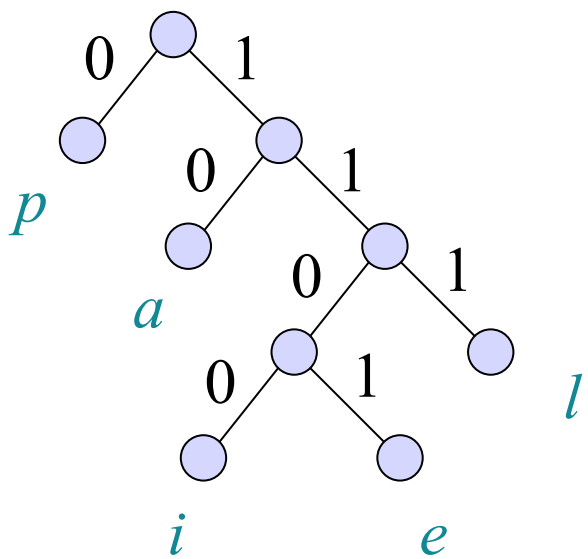
Answer: apple

Prefix Code

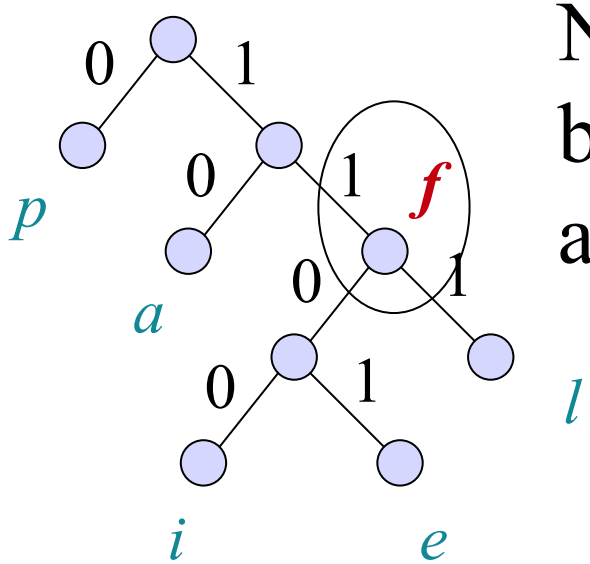
Can check the prefix property by drawing the code tree (for simplicity 0 on the left, 1 on the right)

If every code is located at a *leaf* in the code tree, it is a prefix code

Example: $a = 10$, $p = 0$, $e = 1101$, $l = 111$, $i = 1100$



Example: $a = 10$, $p = 0$, $e = 1101$, $l = 111$, $i = 1100$, $f = 11$



Not a prefix code because ***f*** is not a leaf

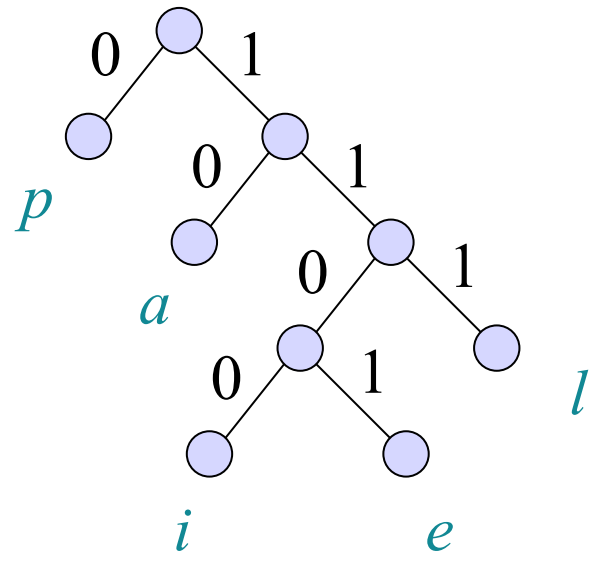
Decoding Prefix Code

Given the code tree,

we can decode a binary sequence in $O(n)$ time

Example: 10001111101

10(a)0(p)0(p)111(l)1101(e)



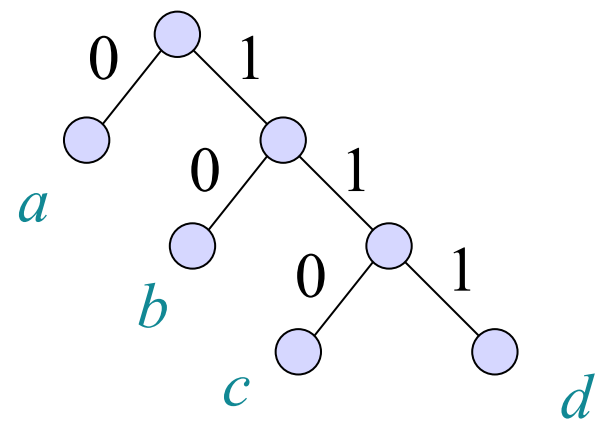
Example

We can use prefix code to shorten the encoding, e.g. we go back to the first example string S = "*ababcaad*"

Previously 2 bits per character for 16 bits

If we set $a = 0$, $b = 10$, $c = 110$ and $d = 111$, then we can encode S using just 14 bits

a	b	a	b	c	a	a	d
0	10	0	10	110	0	0	111



Notice that more frequent letters usually have shorter code but less frequent letters may have longer code

Computational Problem (Draft)

Now consider a general string S, can we find the shortest prefix code to represent S?

Answer: We use ***Huffman code***

Notation

Given a string S, let $f(c)$ be the frequency of the character c in S

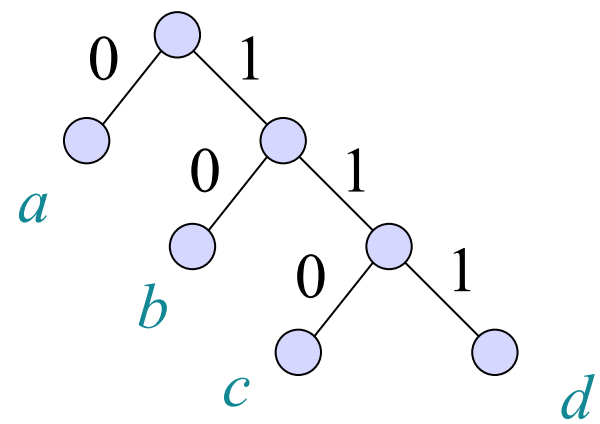
Example: for $S = \text{"}ababcaad\text{"}$,

we have $f(a) = 4$, $f(b) = 2$, $f(c) = 1$, and

$f(d) = 1$

Let T be the code tree corresponding to a prefix code. For each character c, let $d(c)$ be the depth of c in T.

Note: $d(c)$ is also the length of the prefix code for c.



Example: $a = 0$, $b = 10$, $c = 110$ and $d = 111$

$d(a) = 1$, $d(b) = 2$, $d(c) = 3$, and $d(d) = 3$

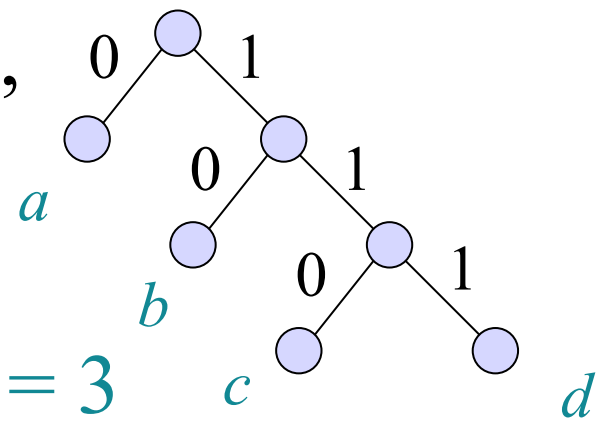
Encoding size $B(T)$

To encode S by the code tree T,

the total number of bits is $B(T) = \sum_{c \in C} f(c)d(c)$

Example:

For S = "*ababcaad*" and $f(a) = 4$, $f(b) = 2, f(c) = 1, f(d) = 1$



For the code tree T and

$d(a) = 1, d(b) = 2, d(c) = 3, d(d) = 3$

$B(T) = 4*1 + 2*2 + 1*3 + 1*3 = 14$

Computational Problem (Formal)

Consider a string S over an alphabet C.

Let $f(x)$ be the number of occurrences of x in S

Find $\mathrm{OPT}(C)$, that is,

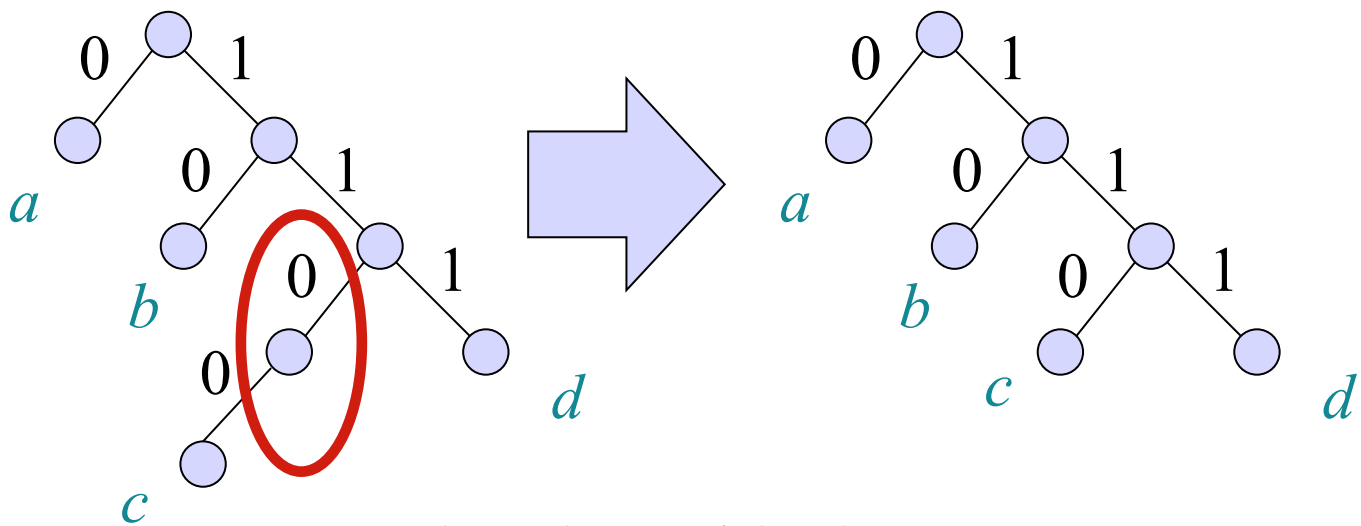
the code tree T that minimizes $B(T) = \sum_{c \in C} f(c)d(c)$

Observation

The optimal code tree must be a *full* binary tree.

Full: every vertex other than the leaves has two children.

If the code tree T' is not a full binary tree, then we can get a better code tree T such that $B(T) < B(T')$ by contracting the vertex with one child.

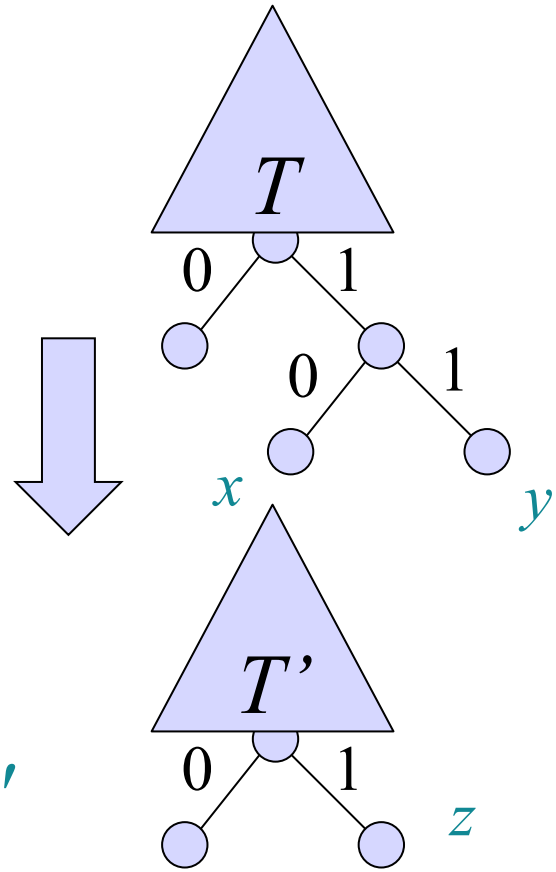


Optimal Substructure

Let T be the optimal code tree for a string S.

Since T is a full binary tree, there exist two leaves x and y which are sibling at the deepest position.

We replace both x and y with another character z and declare the resulting string to be S'. Let T' be the optimal code tree for S'



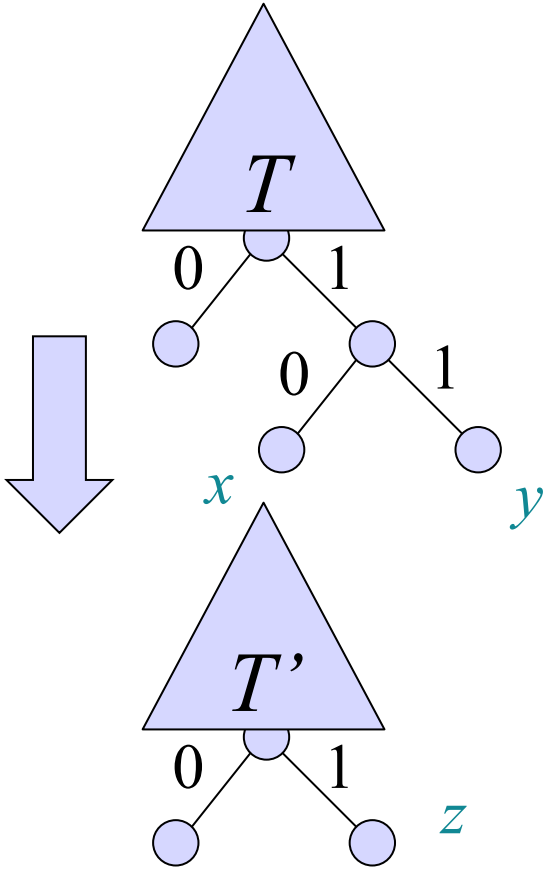
Claim: $B(T) = B(T')+f(x)+f(y)$

We prove the claim by showing:

$B(T) \leq B(T')+f(x)+f(y)$

and

$B(T) \geq B(T')+f(x)+f(y)$



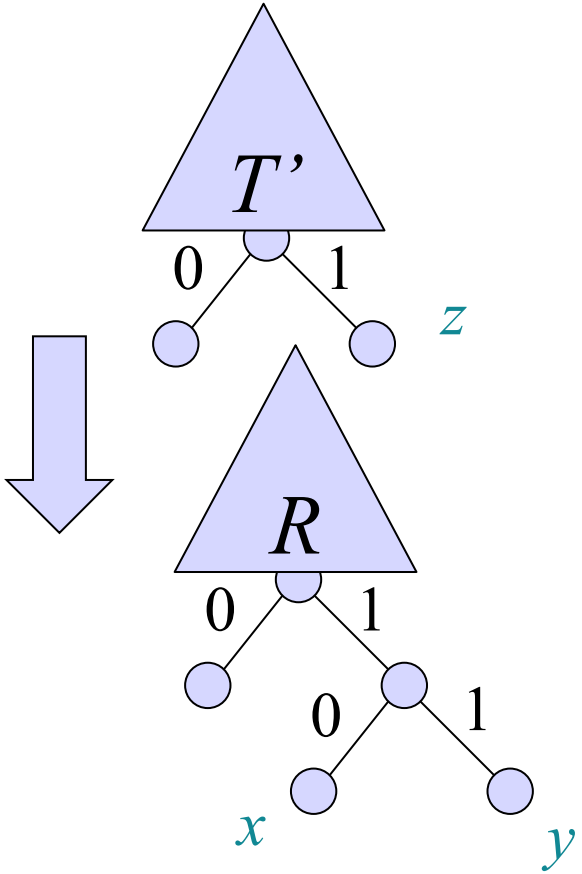
Replace z in T' by a tree with two leaves x and y.

Denote the new tree as R.

$B(R) = B(T')+f(x)+f(y)$

Since T is optimal,

$B(T) \leq B(R) = B(T')+f(x)+f(y)$



Replace the subtree with x and y in T by z

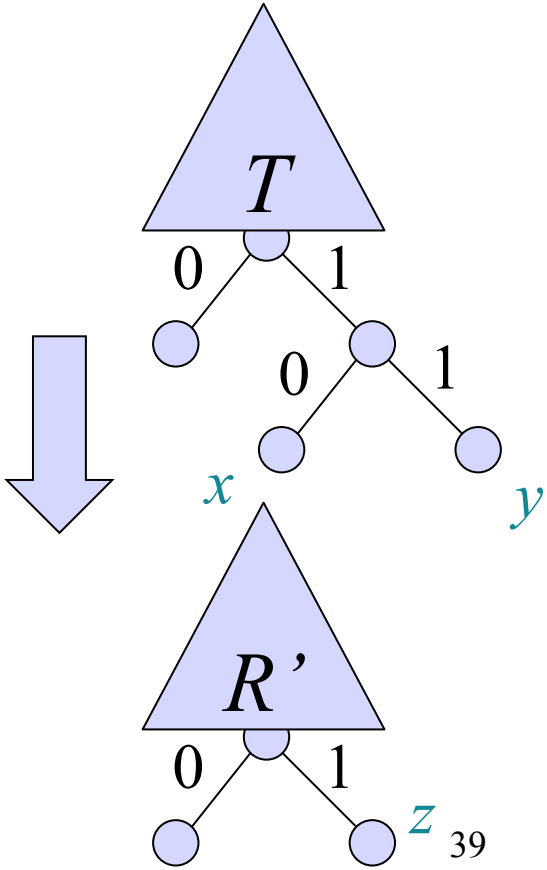
Let the tree be R'.

$B(R') = B(T) - f(x) - f(y) \geq B(T')$

since T' is optimal for S'.

Rewriting we have

$B(T) \geq B(T') + f(x) + f(y)$



Greedy Choice

Let x and y be the two symbols that have the lowest and the second lowest frequencies, respectively

Intuition: Put them deepest in the code tree…

Lemma: There exists an optimal solution where the codewords for x and y have the (same) longest length and differ only in the last bit.

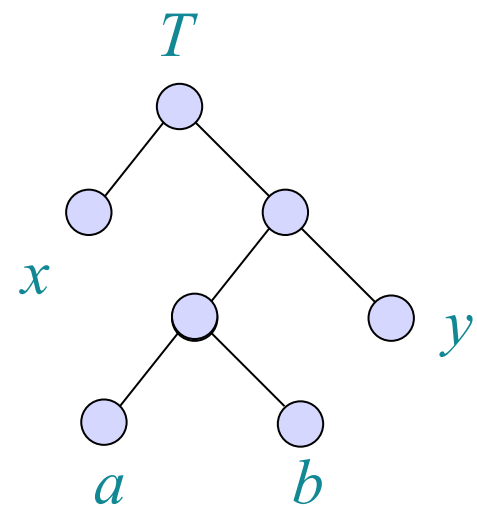
Proof: (*Cut and Paste*)

Let T be an arbitrary optimal code tree.

Let a and b be the two symbols currently with the longest codewords.

Since x and y have the lowest frequencies, we have

$f(x) \leq f(a)$ and $f(y) \leq f(b)$



By swapping x with a, we get the tree T'.

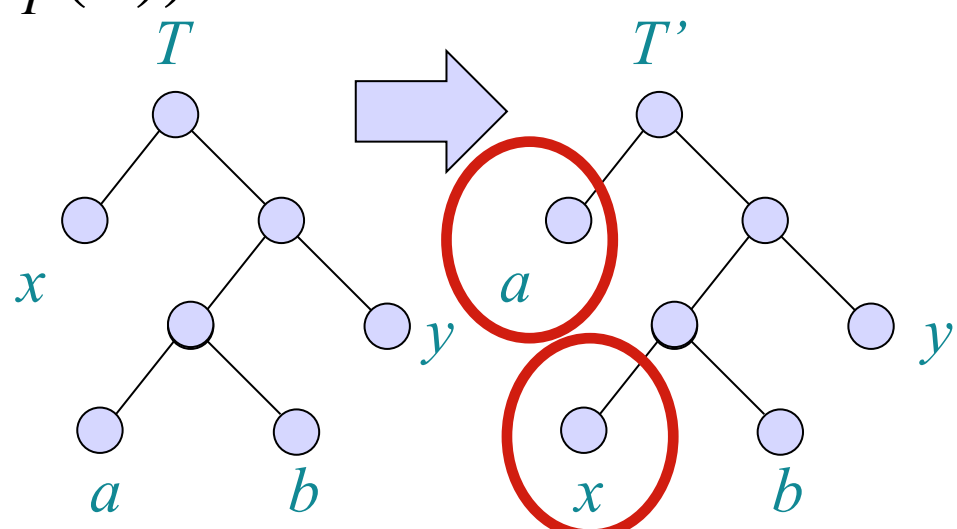
$$B(T) - B(T') = \sum_{c \in C} f(c) d_T(c) - \sum_{c \in C} f(c) d_{T'}(c)$$

$$= f(x) d_T(x) + f(a) d_T(a) - f(x) d_{T'}(x) - f(a) d_{T'}(a)$$

$$= f(x) d_T(x) + f(a) d_T(a) - f(x) d_T(a) - f(a) d_T(x)$$

$$= (f(a) - f(x))(d_T(a) - d_T(x))$$

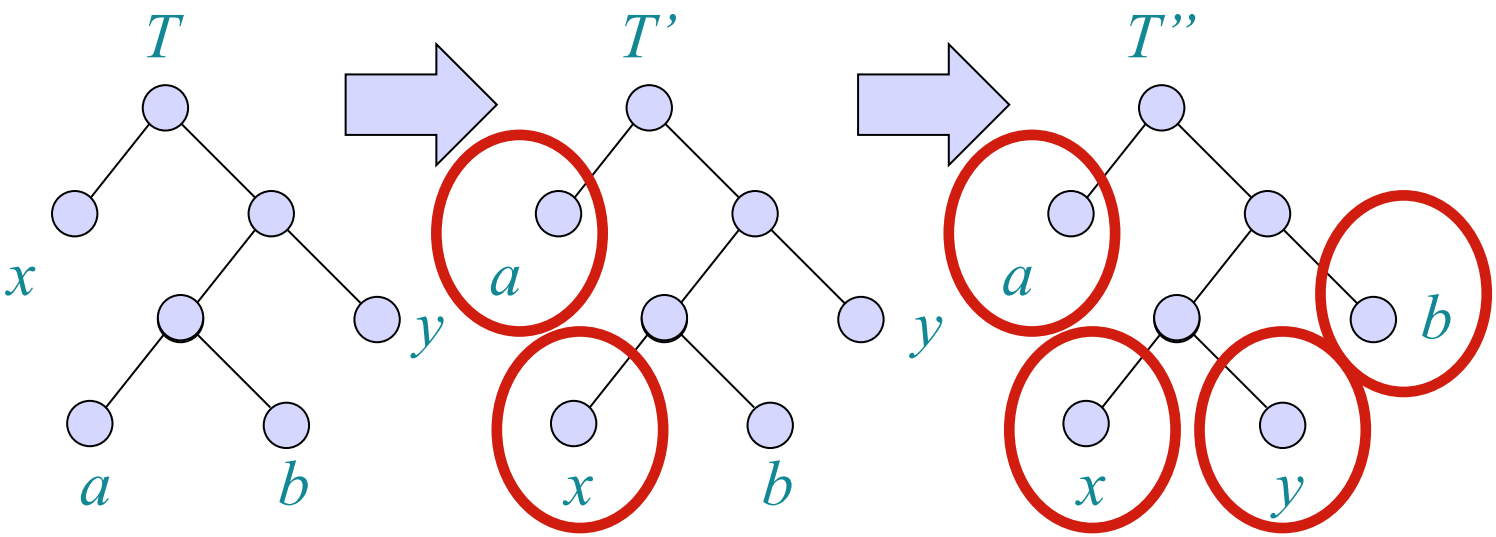
$$\geq 0$$



Hence, $B(T) \geq B(T')$

By swapping y with b, we get the tree T''

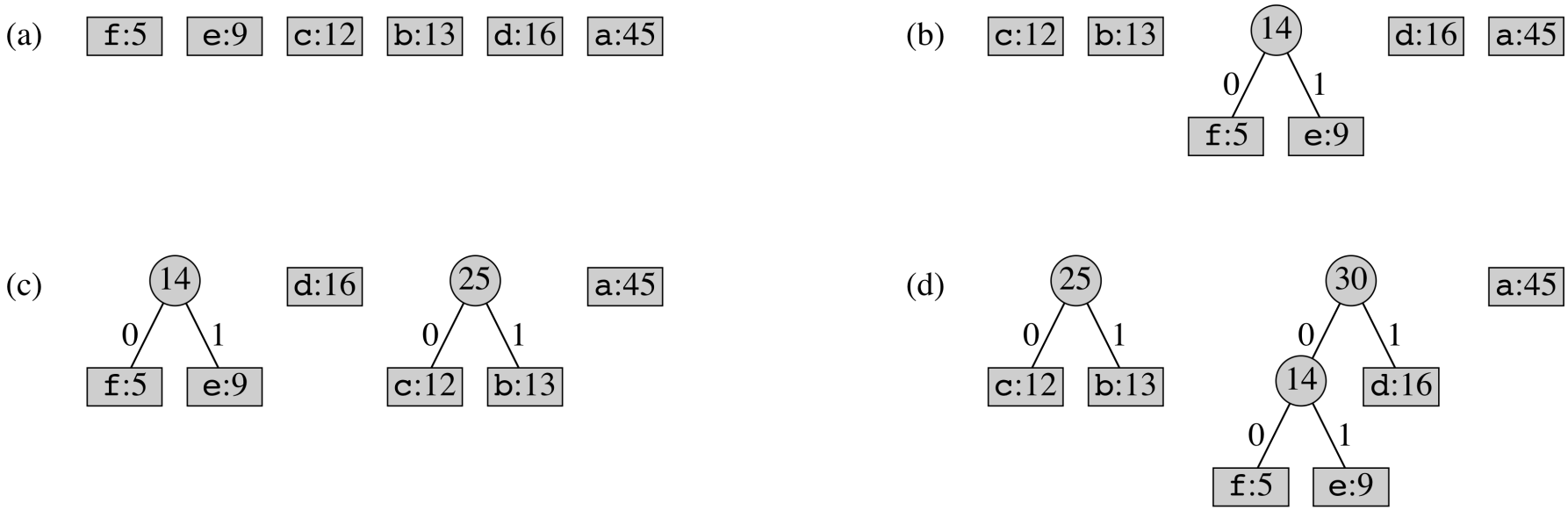
With the same argument as in the previous slide, we can show $B(T') \geq B(T'')$



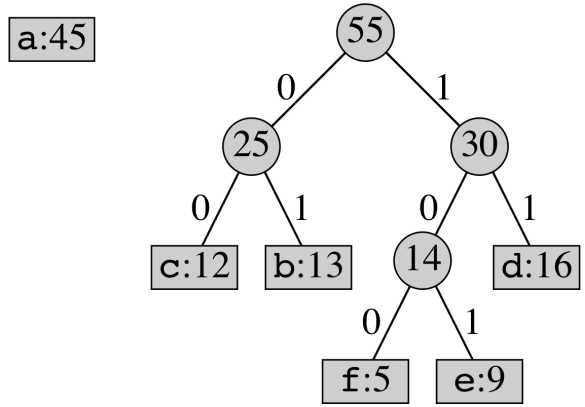
We now have $B(T) \geq B(T') \geq B(T'')$, or $B(T) \geq B(T'')$

Since $B(T) \geq B(T'')$ and T is optimal, we conclude that $B(T) = B(T'')$, i.e. T'' is also an optimal tree in which x and y appear as sibling leaves of maximum depth

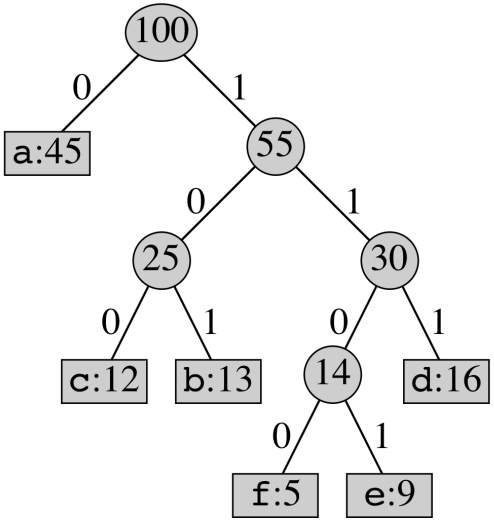
```
HUFFMAN(C)
n ← |C|
Q ← C
for i ← 1 to n – 1
    do allocated a new node z
        left[z] ← x ← EXTRACT-MIN(Q)
        right[z] ← y ← EXTRACT-MIN(Q)
        f[z] ← f[x] + f[y]
        INSERT(Q,z)
return EXTRACT-MIN(Q)        ▷ Return root
```


(a)
f:5
e:9
c:12
b:13
d:16
a:45
(b)
c:12
b:13
14
0
1
f:5
e:9
d:16
a:45
(c)
14
0
1
f:5
e:9
d:16
25
0
1
c:12
b:13
a:45
(d)
25
0
1
c:12
b:13
30
0
1
14
d:16
0
1
f:5
e:9
a:45

(e)


a:45
55
0
1
25
30
0
1
0
1
c:12
b:13
14
d:16
0
1
f:5
e:9

(f)


100
0
1
a:45
55
0
1
25
30
0
1
0
1
c:12
b:13
14
d:16
0
1
f:5
e:9

Recap

- Huffman code – optimal prefix code
- Optimal substructure: $B(T) = B(T')+f(x)+f(y)$
- Greedy choice: there exists an optimal code where the two least frequent symbols have the longest length and have codeword differing only on the last bit
- Greedy algorithm